

Tutorial Lecture 8

Topic: Exact Exponential-Time Algorithms

The following questions are meant to study the material given in the lecture in more depth. Some of those questions could be surprise questions on the exam, some of those questions are only meant for discussion, and yet some are potential research problems. We list those more difficult questions as a possible source for inspiration.

Question 1. Automated Guided Vehicle Lift Scheduling

In this exercise, we look at a simplification of a problem your teacher worked on in his job in industry. In this problem, we need to develop a scheduling algorithm for a lift transporting AGVs (Automated Guided vehicles, i.e., robotic vehicles) between different floors in a large building.

Suppose we have k AGVs with for each AGV a a predetermined floor of origin $f_o(a)$ where the AGV is initially placed and a destination floor $f_d(a)$ where the AGV needs to go to. The lift can transport only one AGV at the same time. The building has r floor levels, numbered from 1 to r , and the lift is initially positioned at floor 1.

The lift can perform the following operations:

- The lift can move to a different floor. If the lift is at floor i , and moves to floor j , this costs $t_0 + t_1 \cdot |i - j|$ time. For simplicity, you may assume t_0 and t_1 are positive integers that are at most 2^k .
- If the lift is empty and currently is at floor $f_o(a)$ for some AGV a , then an AGV a can enter the lift. This costs t_e time.
- If the lift contains AGV a and currently is at floor $f_d(a)$, then AGV a can leave the lift. This costs t_ℓ time.

The problem is to find the fastest sequence of operations for the lift (and the AGVs) such that all AGVs are transported to their respective destination floors.

- a) Show that we can model the above problem as a Travelling Salesman Problem on a graph with $k + 1$ nodes. Conclude that we can solve the lift scheduling problem in $2^k \cdot k^{\mathcal{O}(1)} \cdot r^{\mathcal{O}(1)}$ time using the Held-Karp dynamic programming algorithm.
- b) In practice, each floor has a queue in which all AGVs that are initially on this floor are waiting before entering the lift. Only the AGV at the front of this queue can enter the lift, and all other AGVs in the queue have to wait until they reach the front of the queue (this is different from question (a) where any AGV on the floor can enter the lift).

Show how to modify the Held-Karp dynamic programming algorithm to solve the problem with queues at each floor in $2^k \cdot k^{\mathcal{O}(1)} \cdot r^{\mathcal{O}(1)}$ time.

- c) Because of space restrictions at some floors, the queues introduced in question (b) are placed directly in front of the lift doors at some floors. As a consequence, no AGV can leave the lift at this floor before the AGVs in the queue are transported to their respective destination floors by the lift. This leads to a potential problem: namely, it is now possible that there does not exist any sequence of operations for the lift such that all AGVs are transported to their respective destination floors.

Let $B(i) = 1$ if the queue at floor i blocks AGVs exiting the lift and let $B(i) = 0$ otherwise. Modify your algorithm from question (b) so that it solves the problem with queues that at some floors are placed directly in front of the lift doors in $2^k \cdot k^{\mathcal{O}(1)} \cdot r^{\mathcal{O}(1)}$ time.

- d) Consider the following undirected graph $G = (V, E)$. Let $V = \{a_1, a_2, \dots, a_k\}$ be a set of vertices corresponding to the k AGVs in the lift scheduling problem. Furthermore, let E contain an undirected edge $\{u, v\}$ if and only if u must be transported before v or v must be transported before u due to the restrictions imposed by the queues. That is, u and v belong to the same

queue, or the queue containing v is preventing u from going to its destination floor (or vice versa). Finally, let M be a (maximum) matching in the graph G .

Show that the Held-Karp style algorithm of question (b) can be modified so that it runs in $3^{|M|} 2^{k-2|M|} \cdot k^{\mathcal{O}(1)} \cdot r^{\mathcal{O}(1)}$ time.

Hint: Consider the subsets tabulated by the Held-Karp algorithm and think about which sets are no longer needed.

Open Pool Questions

Lecture 8: Exact Exponential-Time Algorithms

Question 2. *Exact Algorithm for Exact 3-SAT*

Suppose we have a set of Boolean variables $x_1, \dots, x_n$, a set of clauses, where each clause is a subset of $\{x_1, \dots, x_n, \text{not}(x_1), \dots, \text{not}(x_n)\}$, and each clause has at most three literals (elements). In the EXACT 3-SATISFIABILITY, we want to determine if there exists a truth assignment such that each clause has *exactly* one true literal.

For instance, if we have clauses $\{x_1, x_2, x_3\}$, $\{\text{not}(x_1), x_2, x_4\}$, then setting x_1 to true, x_2 to false, x_3 to false, and x_4 to true is a solution. Also, setting x_1 and x_2 to true does not lead to a solution, as the first clause has two literals that are true.

1. Show that we can create a reduction rule using polynomial time that transforms an instance of EXACT 3-SATISFIABILITY into an equivalent smaller instance of EXACT 3-SATISFIABILITY where all clauses have size two or three. (Smaller means, not increasing the number of variables or clauses)
2. Show that we can create a reduction rule using polynomial time that transforms an instance of EXACT 3-SATISFIABILITY into an equivalent smaller instance of EXACT 3-SATISFIABILITY where all clauses have size exactly three. (Smaller means, not increasing the number of variables or clauses)
3. Give an exponential-time algorithm that solves the EXACT 3-SATISFIABILITY problem. Analyse the time of your algorithm. Your algorithm should use $\mathcal{O}^*(c^n)$ time for some $c < 2$.

Question 3. *Bisection Width*

Consider the following problem.

Given: Graph $G = (V, E)$, with $|V|$ even.

Question: Find a partition of V into two sets V_1, V_2 , with $|V_1| = |V_2| = |V|/2$, such that the number of edges with one endpoint in V_1 and one endpoint in V_2 is as small as possible.

This problem is NP-complete, and an $\mathcal{O}^*(2^n)$ algorithm is trivial. Find an efficient exact algorithm for this problem with running time $\mathcal{O}^*(c^n)$ for some $c < 2$.

Homework Lecture 8

Hand-In: Tutorial 10, Tuesday March 10th

Please prepare the following homework question for the Hand-In date. Please bring your homework on paper. You get one point for handing in the homework and you get another point for participating in a round of peer-feedback. Taking place during the tutorial.

Consider the following algorithm for the INDEPENDENT SET problem on graphs of maximum degree three. (If we remove a vertex, we also remove all edges incident to the vertex.)

- If G has a vertex v of degree 0, then recursively find a solution for the graph, obtained by removing v and add v to this solution.
- If G has a vertex v of degree 1, then let H be obtained by removing v and its neighbour of G . Recursively solve the problem for H , and then add v to the found solution.
- If G has a vertex v of degree 2, and this vertex is adjacent to another vertex w of degree 2, then let H be obtained by removing v and w , and adding an edge between the other neighbour of v , say x , and the other neighbour of w , say y . removing v and its neighbour of G . Recursively solve the problem for H , and then add one of v or w to the found solution.
- If G has a vertex v of degree 2, and the neighbours of v are adjacent, then let H be obtained by removing v and its neighbour of G . Recursively solve the problem for H , and then add v to the found solution.
- If all vertices in G have degree two, then solve G optimally. (This can be done easily in linear time, see part (ii).)
- Otherwise, take a vertex v of degree exactly three. Compute the two following solutions, and return the largest one:
 - Recurse on the graph, obtained by removing v .
 - Recurse on the graph, obtained by removing v and its neighbours. Add v to the found solution.

(i) Argue that this is a correct algorithm for the INDEPENDENT SET problem on graphs of maximum degree three. (Hint: look at each step of the algorithm separately.)

(ii) How do we solve the problem if all vertices have degree exactly two?

(iii) Let G be a graph with maximum degree three, and n_3 vertices of degree three. Let v be a vertex of degree three. Let H be obtained from G by removing v , and then applying all reduction rules of the algorithm exhaustively (all 'if' cases, but not the 'otherwise' case). Argue that H has at most $n_3 - 4$ vertices of degree three.

(iv) Let G be a graph with maximum degree three, and n_3 vertices of degree three. Let H' be the graph, obtained from G by removing v and its neighbours. Argue that H' has at most $n_3 - 4$ vertices of degree three.

(v) Show that the algorithm uses $\mathcal{O}^*(2^{n/4})$ time.

(vi) **Extra (not part of the homework):** Can you give a sharper time bound for the algorithm? (This can be quite some work)